CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation

Problem Statement: A product team classifies customer reviews as positive or negative. It must pick the best recurrent model and its settings within a fixed compute budget. The IMDB movie-review sentiment dataset (or product reviews from an e-commerce site). The objective is to tune RNN, LSTM and GRU models automatically with KerasTuner. Use TensorFlow, Keras, NumPy, KerasTuner, Matplotlib.

Tasks:

1. Tokenise and pad the reviews; add an embedding layer.

2. Define a model-builder function that chooses the cell type (RNN, LSTM, GRU).

3. Tune units, layers, dropout, learning rate and embedding size with KerasTuner.

4. Report the best configuration for each cell type.

5. Compare accuracy, parameter count and training time.

FORMULA:

1. Embedding Layer

Output = Embedding(Input Sequence, Embedding Dimension)

2. RNN Hidden State

$h(t) = \tanh(W(hh) \times h(t-1) + W(xh) \times x(t) + b(h))$

3. LSTM Cell State

$C(t) = f(t) \times C(t-1) + i(t) \times \tilde{C}(t)$

4. GRU Hidden State

$h(t) = (1 - z(t)) \times h(t-1) + z(t) \times \tilde{h}(t)$

5. Accuracy

Accuracy = (Correct Predictions / Total Predictions) × 100

6. Loss Function (Binary Cross-Entropy)

$Loss = -[y \times \log(p) + (1-y) \times \log(1-p)]$

7. Parameter Count

Total Parameters = Trainable Parameters + Non-Trainable Parameters

8. Training Time

Training Time = End Time - Start Time

9. Dropout

Output = Input × Mask

10. Learning Rate Update

New Weight = Old Weight - Learning Rate × Gradient

CODE:

```
import time

import numpy as np

import tensorflow as tf

import keras_tuner as kt
```

```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models
np.random.seed(42)
tf.random.set_seed(42)
# Task 1: Load, tokenize and pad the IMDB reviews
vocab_size = 5000
max_length = 150
(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=vocab_size
)
x_train = pad_sequences(
    x_train, maxlen=max_length, padding="post", truncating="post"
)
x_test = pad_sequences(
    x_test, maxlen=max_length, padding="post", truncating="post"
)
# Use a smaller training subset to reduce execution time
x_train = x_train[:8000]
```

```
y_train = y_train[:8000]
x_val = x_test[:2000]
y_val = y_test[:2000]
# Task 2 and 3: Define the model builder
def build_model(hp, cell_type):
    model = models.Sequential()
    embedding_dim = hp.Choice(
        "embedding_dim", [32, 64]
    )
    model.add(layers.Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        mask_zero=True
    ))
    units = hp.Choice("units", [32, 64])
    num_layers = hp.Choice("num_layers", [1, 2])
    dropout_rate = hp.Choice("dropout", [0.2, 0.4])
    for i in range(num_layers):
        return_sequences = i < num_layers - 1
        if cell_type == "RNN":
```

```
        layer = layers.SimpleRNN(
            units,
            return_sequences=return_sequences,
            dropout=dropout_rate
        )
    elif cell_type == "LSTM":
        layer = layers.LSTM(
            units,
            return_sequences=return_sequences,
            dropout=dropout_rate
        )
    else:
        layer = layers.GRU(
            units,
            return_sequences=return_sequences,
            dropout=dropout_rate
        )
    model.add(layer)
model.add(layers.Dense(1, activation="sigmoid"))
learning_rate = hp.Choice(
```

```
        "learning_rate", [0.001, 0.0005]
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Task 4 and 5: Tune and compare each recurrent model
results = []
for cell_type in ["RNN", "LSTM", "GRU"]:
    print("\nTuning", cell_type)
    tuner = kt.RandomSearch(
        hypermodel=lambda hp: build_model(hp, cell_type),
        objective="val_accuracy",
        max_trials=4,
        executions_per_trial=1,
        directory="imdb_tuning",
```

```
    project_name=cell_type,
    overwrite=True
)
start_time = time.time()
tuner.search(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=3,
    batch_size=64,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=1,
            restore_best_weights=True
        )
    ],
    verbose=1
)
training_time = time.time() - start_time
best_hp = tuner.get_best_hyperparameters(1)[0]
```

```
    best_model = tuner.get_best_models(1)[0]
    loss, accuracy = best_model.evaluate(
        x_val, y_val, verbose=0
    )
    results.append({
        "Model": cell_type,
        "Accuracy": accuracy,
        "Parameters": best_model.count_params(),
        "Time": training_time,
        "Hyperparameters": best_hp.values
    })

    print("Best hyperparameters:", best_hp.values)
    print("Validation accuracy:", round(accuracy, 4))
    print("Parameter count:", best_model.count_params())
    print("Training time:", round(training_time, 2), "seconds")
# Display comparison
print("\nFINAL MODEL COMPARISON")
for result in results:
    print(
```

```
        result["Model"],
        "| Accuracy:", round(result["Accuracy"], 4),
        "| Parameters:", result["Parameters"],
        "| Time:", round(result["Time"], 2), "seconds"
    )
# Plot accuracy comparison
names = [r["Model"] for r in results]
accuracies = [r["Accuracy"] for r in results]
plt.figure(figsize=(7, 4))
plt.bar(names, accuracies)
plt.xlabel("Recurrent Model")
plt.ylabel("Validation Accuracy")
plt.title("RNN vs LSTM vs GRU Accuracy")
plt.ylim(0, 1)
plt.show()
# Plot training time comparison
times = [r["Time"] for r in results]
plt.figure(figsize=(7, 4))
plt.bar(names, times)
plt.xlabel("Recurrent Model")
```

```
plt.ylabel("Tuning Time (seconds)")

plt.title("Hyperparameter Tuning Time")

plt.show()
```

OUTPUT:

```
Trial 4 Complete [00h 00m 07s]

val_accuracy: 0.8230000138282776


Best val_accuracy So Far: 0.8240000009536743

Total elapsed time: 00h 00m 33s

/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 20 variables.

  saveable.load_own_variables(weights_store.get(inner_path))

Best hyperparameters: {'embedding_dim': 32, 'units': 32, 'num_layers': 2, 'dropout': 0.4, 'learning_rate': 0.001}

Validation accuracy: 0.824

Parameter count: 172705

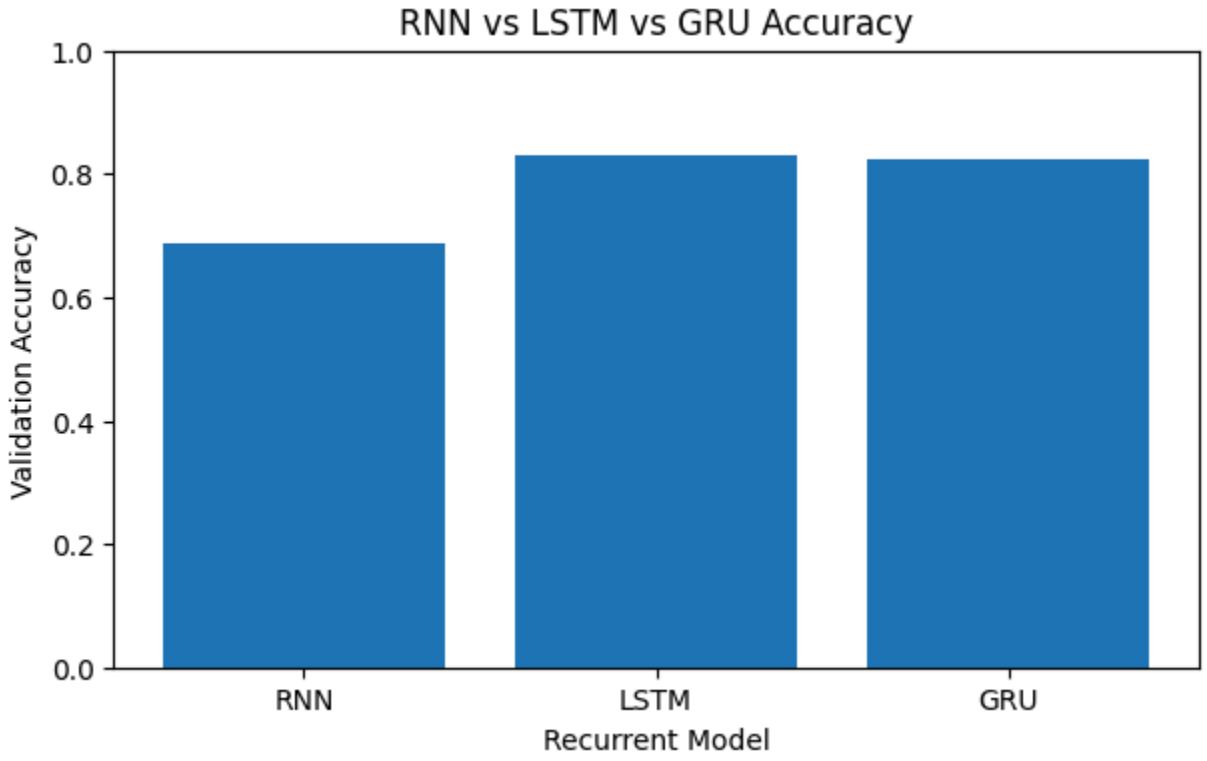
Training time: 33.44 seconds

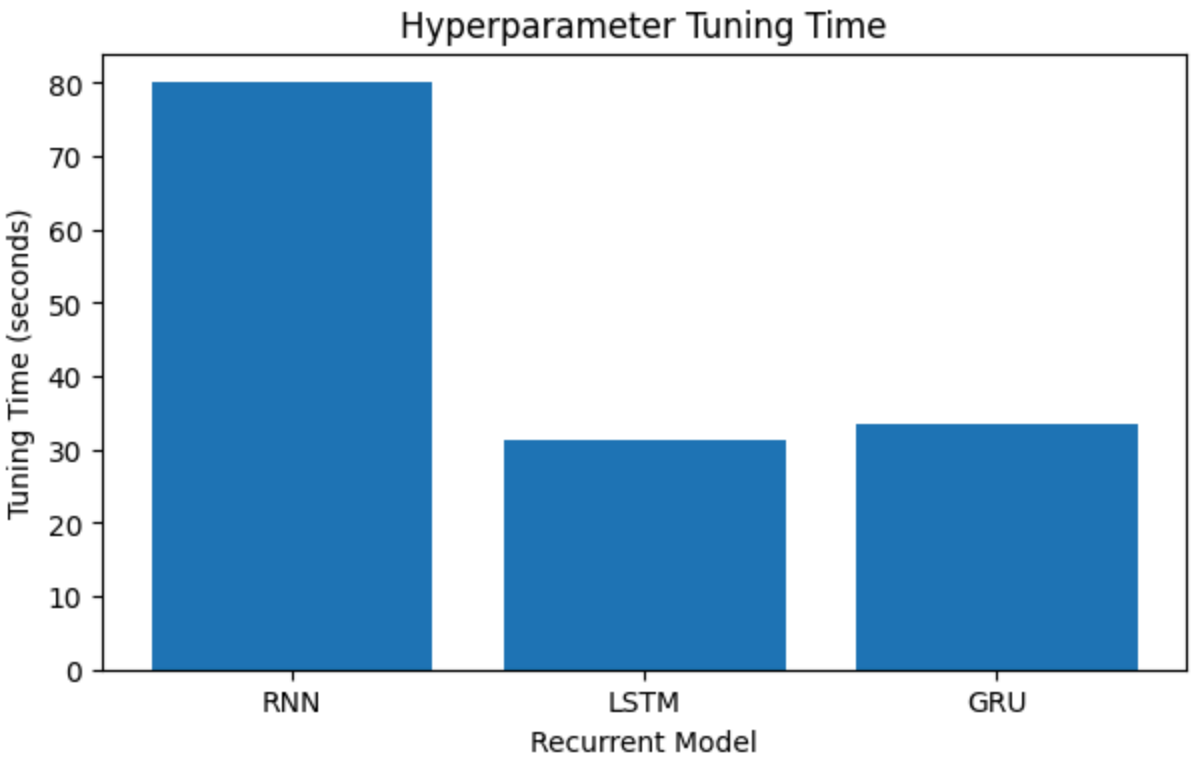

FINAL MODEL COMPARISON

RNN | Accuracy: 0.6885 | Parameters: 328321 | Time: 80.03 seconds
```

LSTM | Accuracy: 0.83 | Parameters: 353089 | Time: 31.4 seconds

GRU | Accuracy: 0.824 | Parameters: 172705 | Time: 33.44 seconds




Hyperparameter Tuning Time
Tuning Time (seconds)
0
10
20
30
40
50
60
70
80
RNN
LSTM
GRU
Recurrent Model